

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Emily Nguyen* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

The extract -> transform -> summarize -> cite task is a Sequential Pipeline because the order is fixed in code and no agent decides what comes next at runtime. The one where the next step depends on prior results is a Supervisor-Router: it looks at the current state after each turn and routes to whichever agent should go next. Centralized control trades flexibility for predictability because one controller owns the ordering, so there is a single place you can inspect when something breaks.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

A fourth worker only pays for the coordination tax if it buys real parallelism, or handles a job that needs a conflicting prompt the other three can't share -- you earn the agent by naming how three workers actually failed, not by wanting more. The failure mode I'd be inviting is bad decomposition: the orchestrator now has to re-split the task into four pieces that can overlap or leave gaps, and aggregation can hide a worker that quietly returned a wrong answer.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

Exposing capabilities through a shared MCP protocol means each tool is defined once behind a standard interface instead of being hand-wired and separately maintained inside every worker, so a fix lands everywhere at once. A FastMCP server lets you swap or version the implementation behind that same interface, so none of the workers need rewriting when a tool changes. That keeps the tools as shared infrastructure, separate from the controller that owns the flow.

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Rohan Patel* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

Fixed sequence = Sequential Pipeline, since you wire the order in code.

Unpredictable next step = Supervisor-Router, because the supervisor re-decides each turn based on the latest state, kind of like ReAct at the system level.

Centralized control is more predictable and debuggable because a single named component decided the order, so there is one place to look.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

The fourth agent is worth it only if something specific materializes -- like independent verification, or domain-specific tooling the others shouldn't carry -- otherwise you're just paying more latency and delegation calls for nothing. The failure mode I invite is lossy aggregation: with four outputs to merge, the orchestrator can drop or blur part of a worker's result.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

A shared MCP tool protocol means the capability is written once and every worker calls it the same way, instead of duplicating drifting tool code in each agent. FastMCP gives you a clean seam -- workers depend on the interface, so you can version the tool without touching the workers.

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Marcus Johnson* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

The extract-transform-summarize-cite one is a Sequential Pipeline because it always runs in the same order. The other one is a Supervisor-Router since you don't know the next step ahead of time. Centralized control is easier to debug because everything goes through one controller.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

You'd add the fourth worker if it makes the system faster by running things in parallel. The failure mode is that it might not work right and give a wrong answer.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

Using a shared MCP protocol is better because you don't have to wire the tool into every agent separately, you just define it once. FastMCP lets you update the tool without breaking the workers.

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Sofia Garcia* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

The fixed extract -> transform -> summarize -> cite chain is a Sequential Pipeline; the order is set in the code and nothing chooses it at runtime. The task where the next sub-task depends on prior results is a Supervisor-Router, because the supervisor inspects the state and routes the next turn. Centralized control gives up flexibility but you get predictability since one controller owns the flow and you can trace its decisions.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

A fourth worker is only worth the coordination tax if it adds genuine parallelism or a capability the other three can't cover. The failure mode is cascading failure -- if the new worker breaks, its bad output can flow downstream into the aggregation.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

Shared MCP tools matter because several workers need the same capabilities, and defining them once behind a protocol avoids maintaining copies in each agent.
FastMCP helps you swap a tool version out when you need to.

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Daniel Kim* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

The first task is a Sequential Pipeline. The second one, where the next step depends on the results, is an Orchestrator-Worker because the orchestrator decides what to do next. Centralized control is more predictable because there's one controller running it.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

Adding a fourth worker would be worth it if it can do more work and help the system. A failure mode is that adding agents makes it more complex.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

A shared MCP protocol is a standard way for agents to get tools so they all use the same setup. FastMCP is a server that hosts the tools so you can change them.

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Katie O'Brien* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

Extract -> transform -> summarize -> cite is a Sequential Pipeline because it's a fixed chain wired in code. The unpredictable one is a Supervisor-Router, which routes each turn from the current state. Centralized control trades flexibility for predictability and debuggability because a single controller owns the ordering -- one place decided it, so one place to inspect when it misbehaves.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

The fourth worker only justifies the tax if it delivers something concrete, like parallelism from an independent sub-task that now runs at the same time. By adding it I'm inviting over-decomposition -- splitting the task into more pieces than it needs, which can overlap or miss the goal.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

Going through a shared MCP protocol means you define the capability once and every worker calls it the same way instead of hand-wiring it into each one.

FastMCP lets you version a tool behind the interface so workers keep calling it the same way.

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Jamal Williams* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

The fixed sequence is a Sequential Pipeline. The one that depends on prior results is a Supervisor-Router. Centralized means one controller runs the show, which makes it predictable.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

You'd only add the fourth worker if it runs a separate task in parallel and actually speeds things up. The failure I'd invite is underspecified sub-tasks -- the orchestrator might hand it a fuzzy job it can't do well.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

It's better to use a shared MCP protocol so you're not rebuilding the same tool inside every worker. FastMCP means you can swap the tool implementation and the workers still call it the same way through the interface.

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Elena Rossi* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

The extract -> transform -> summarize -> cite job is a Sequential Pipeline -- the order is fixed in code with no routing at runtime. The task whose next step depends unpredictably on prior output is a Supervisor-Router, which re-decides the next agent each turn from the state. Centralized control trades flexibility for predictability because one named component owns the control flow, so there is one place that decided the order and one place to debug.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

A fourth worker only earns its keep if a specific benefit shows up, like independent verification of the other workers' output -- you don't add it just because you want to. The failure mode I invite is lossy aggregation, where merging four results drops or distorts part of what a worker produced.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

<i>A shared MCP protocol defines each capability once behind a standard interface so the workers don't each carry their own drifting copy. FastMCP buys a clean seam for change when you swap a tool.</i>

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Grace Thompson* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

The first one is a Sequential Pipeline because the steps are in order. The second is a Supervisor-Router.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

A fourth agent would be worth it if it helps the system do the task better. A failure mode is that it could be wrong or slow.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

MCP is a standardized way for agents to use tools. FastMCP is a framework for building MCP servers so you can make tools easily.

Quiz — Week 9, Class 1

Centralized Multi-Agent Patterns

Name: *Yusuf Ahmed* Date: *Mar 12, 2026*

Short answer — 2–4 sentences each. Explain your reasoning and how the idea applies to agentic design; definitions alone will not score.

1. Module 9 catalogs the centralized patterns — Orchestrator-Worker, Hierarchical, Supervisor-Router, Sequential Pipeline. Take a task that is a genuinely *fixed* sequence (extract → transform → summarize → cite) and one where the next sub-task *depends unpredictably* on prior results, and say which centralized pattern fits each — then explain why centralized control trades flexibility for predictability and debuggability.

I think the fixed one is an Orchestrator-Worker and the changing one is a Hierarchical pattern because the boss agent manages the others. Centralized just means it's all controlled in one place.

2. Module 9 warns about the **coordination tax** — every added agent multiplies delegation calls, latency, failure modes, and debugging difficulty. You are tempted to add a fourth worker to an orchestrator-worker system. Explain what specific benefit would have to materialize for that fourth agent to be worth the tax, and name a failure mode (e.g., underspecified sub-tasks, lossy aggregation, cascading failure) you would be inviting by adding it.

Adding another worker is good because more agents can get more done, so the fourth one would help finish faster. A downside is it costs more.

3. Primers 19 and 20 present MCP and FastMCP as the standardized way agents get tools. In a centralized multi-agent system where several workers each need overlapping capabilities, explain why exposing those capabilities through a *shared MCP tool protocol* (rather than hand-wiring tools into each agent) matters — and what a FastMCP server buys you when you want to swap or version a tool without rewriting every worker.

MCP lets agents share tools so you don't have to build them again. FastMCP is faster.